

문 1) 스레싱

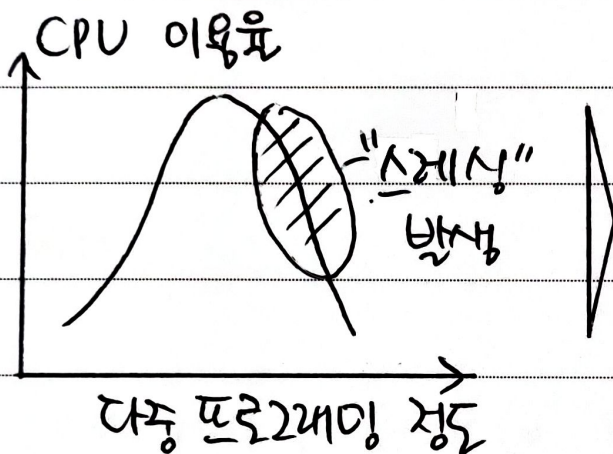
답)

1. Page Fault 과도, 스레싱의 정의

- 병렬 프로그래밍 환경에서 과도한 병렬화로 Page Fault가 많아져 CPU 실행률 급감 현상

2. 스레싱의 개념도

1) 스레싱의 개념도



스레싱 현상

- CPU 이용률 급감
- Page Fault 급증
- Process 부하 발생

- 스레싱 발생 시 CPU 성능 감소하므로 해결 필요

2) 스레싱의 해결 방안

방안	개념도	설명
PFF		• Page Fault rate <u>Frame 수</u> 간 <u>균형</u> • 하한 ~ 상한 사이
Working Set	0 1 [2 1 3] 2 ↓ WS = [1, 2, 3] Δ = 3	• <u>자주</u> 사용하는 page 를 <u>메인 메모리 상주</u> • <u>지역성</u> 활용 방안

- 이 외에도 다중 Programming 부하 감소 방안 있음

3. 다중 프로세싱 부하 감소 방안

Timeout 적용	Thread 사용	Stack Point
- 특정 Process의 CPU 독점 방지 - Timer 적용	- 여러 Process - 실행 시 CPU 부하 계감	Process 간 전환 시 <u>단순 Pointer만 이동</u>
- 스케줄 및 다중 Process 부하 분산해 CPU 성능 향상 13'08" "끝"		

문 2) BGP

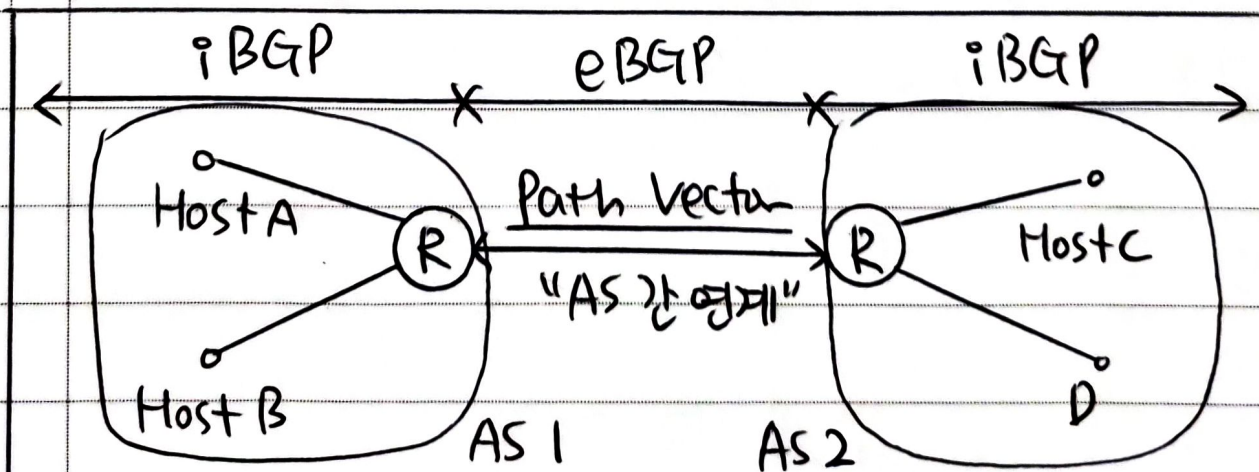
답)

1. TCP 119, BGP의 정의

- 서로 다른 AS 간 연결을 iBGP, eBGP 기반으로 TCP 119 port 통해 수행하는 Unicast Protocol.

2. BGP의 구성도 및 설명

1) BGP의 구성도



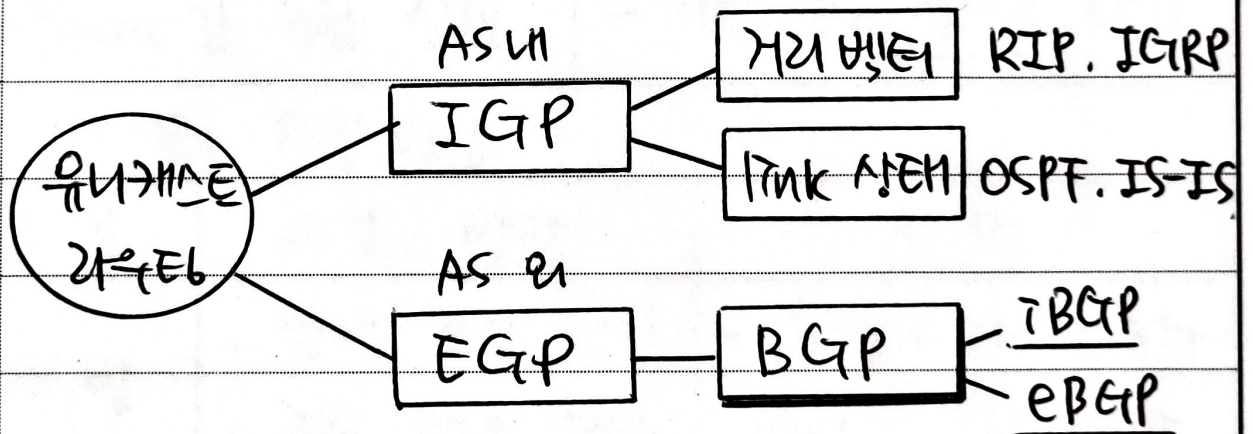
- BGP는 iBGP와 eBGP 기반으로 path vector.

2) BGP 의 종류

구분	개념도	설명
eBGP		- AS "간" data 전송 - Neighbor <u>수동</u> 설정 <u>Path Vector</u> 방식
iBGP	<AS 내부>	- AS '내부' 전송 <u>TTL 225</u> <u>TCP Handshake</u>

- BGP 는 유니캐스트 라우팅 프로토콜의 한 종류.

3. BGP 의 유니캐스트 프로토콜 종류



- NW 3계층의 IP 패킷 전송 시 사용 프로토콜.

13'19" "끝"

문 1) ① 교착상태 발생 조건, 해결방안 ② Banker's Algorithm ③ wait-die, wound-wait

답)

1. Process 실행 불가, 교착상태 발생 조건 / 해결방안

1) 교착상태 발생 조건 (And 조건)

① 현형 대기

- 자원 graph로 확인

② 비선형 스케줄링

- FCFS, SJF, HRN

③ 상호 배제

- Process 간 독립 실행

④ 정유라 대기

- 자원 정유 상태로 대기

2) 교착상태 해결 방안

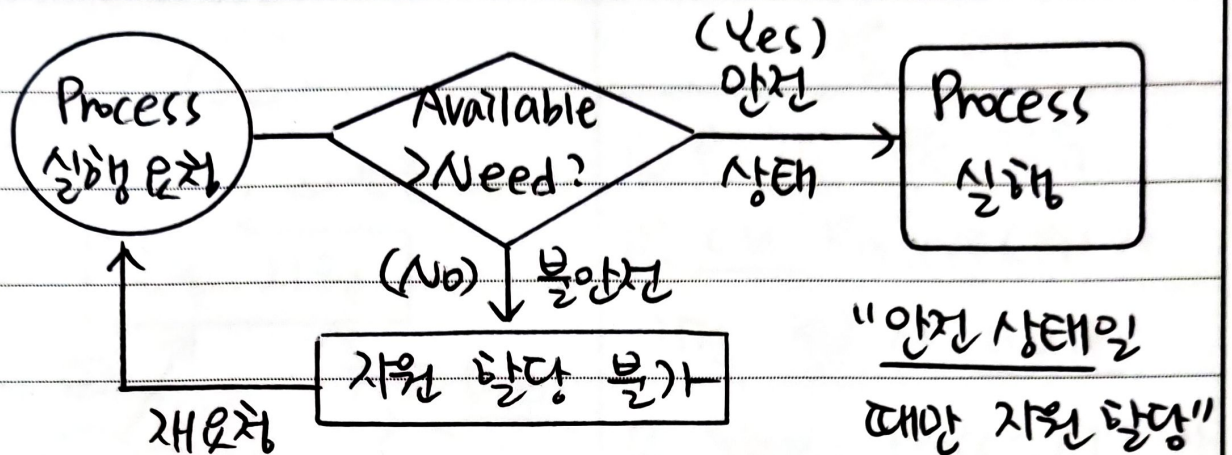
구분	해결 방안	설명
예 방	• 발생 조건 방지 • 조건 발생 예방	- 현형 대기, 정유라 배제 등 조건 예방
<u>현 띠</u>	• <u>Banker's Algorithm</u> • <u>Wait-Die</u>	- 안전 상태 검사 - <u>Wound wait</u> 등
부 구	• Process 중지 • <u>선형 스케줄링</u>	- 자원 <u>정유 해제</u> - <u>RR</u>, <u>SRT</u> 등
검 출	• <u>자원 그래프</u> • <u>Timeout 검출</u>	- 자원, Process 강제 - 실행 불가 시 검출

- "현 띠" 알고리즘을 활용함 deadlock 해결이 많음

2. 안전 상태 예시, Banker's Algorithm

1) Banker's Algorithm 개념 및 매커니즘

개념 • Process 별 필요한 자원을 계산해
"안전 상태일" 때만 자원 할당하는 알고리즘



- 자원 구조 4가지를 바탕으로 안전 상태를 파악.

2) Banker's Algorithm 자원구조

자원 구조	역할	설명
Available	• 각 Process에 할당 가능한 자원량	- $(Available > Need)$ 일 시 자원 할당
Max	• Process 별 최대 필요한 자원량	- Max 충족 되어야 Process 실행 가능
Allocation	• Process 별 할당 받은 자원량	- 실제로 현재 Process가 할당 받은 자원
Need	• Process 별 추가로 필요한 자원량	- $Need = (Max - Allocation)$

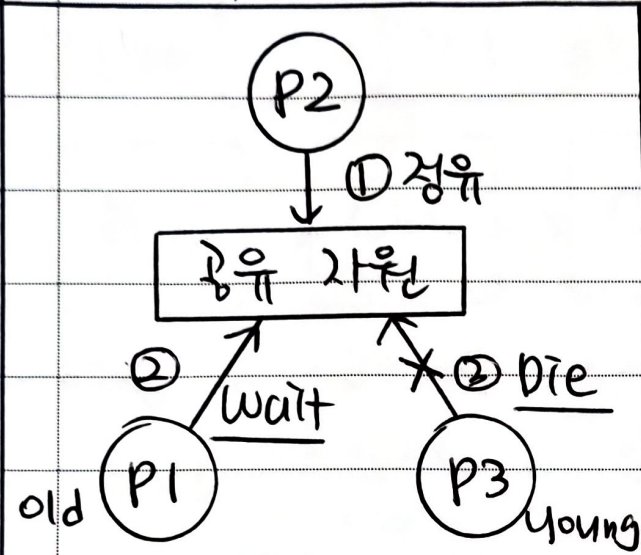
- Banker's Algorithm 은 극히 복잡하여
wound-wait, wait-die 등의 기법 사용.

3. 교착 상태 회피. wait-die or wound-wait

1) 비선점 방식. wait-die 설명

개념

- Old Process 는 wait, 우선순위가 높으면 die 하는 비선점 교착 회피 방안



설명

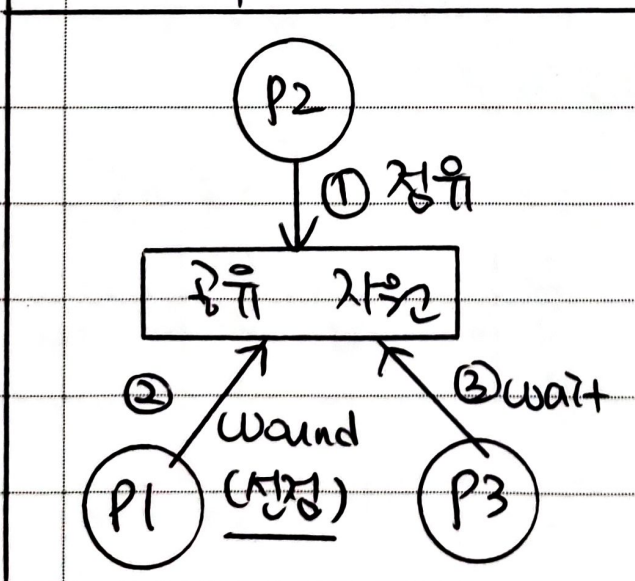
- ① P2가 자원 점유
- ② Old Process (P1)가 자원 접근 시 wait
- ③ Young Process (P3)가 자원 접근 시 die

- Young Process Die 시 Timestamp 설정 후 재시작해 Starvation 방지. 선점 방식도 사용.

2) 선점 방식. wound-wait 설명

개념

- 우선순위가 높은 process 가 자원 선점 허용해 교착 상태 회피하는 선점 기법



설명

- ① P2가 자원 점유
- ② P1이 P2 자원을 빼앗아 "선점" 후 실행
- ③ P3는 P2 실행을 wait 한 후 실행

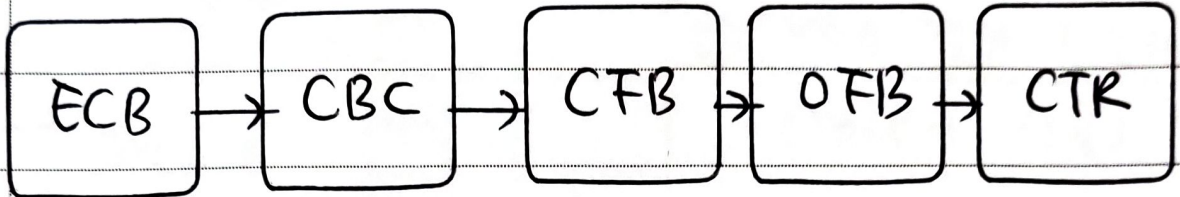
- 무한 교착 상태 찾아 CPU 부하 발생하여 고려 필요.

문 2) 블록 암호화 운영 모드

답)

1. 대칭키 기반, 블록 암호화 운영 모드 개요

블록 암호화	• 가변 길이 문자열을 <u>고정 크기 블록</u> 으로 나눠 각각 암호화 후 <u>결합하는</u> 암호화
-----------	--

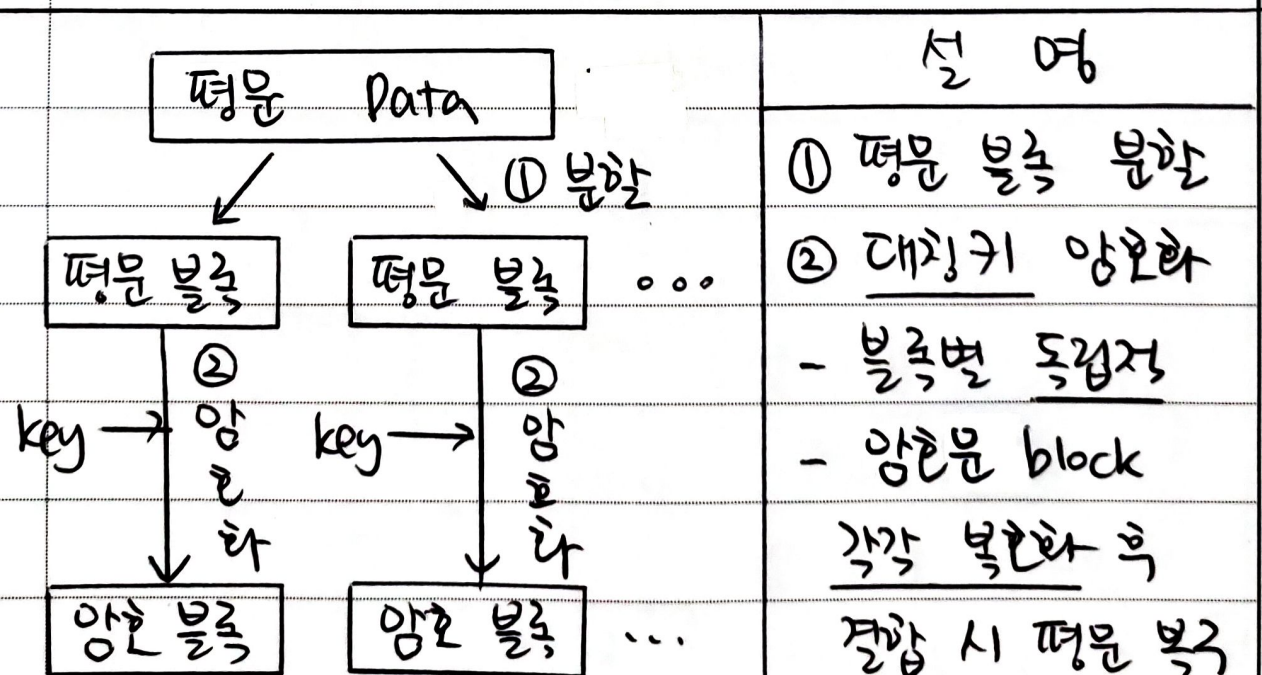


• 간단 • 독립적	• IV 사용 • 암호화 chain	• 스트림 암호화 • Feedback	• IV를 암호화 Feedback	• Feedback 값 • Counter
---------------	------------------------	----------------------------	--------------------------	------------------------------

- 각 운영모드는 암호화/복호화 순서 등 상이함.

2. 블록 암호화 운영 초기 모드 - ECB, CBC

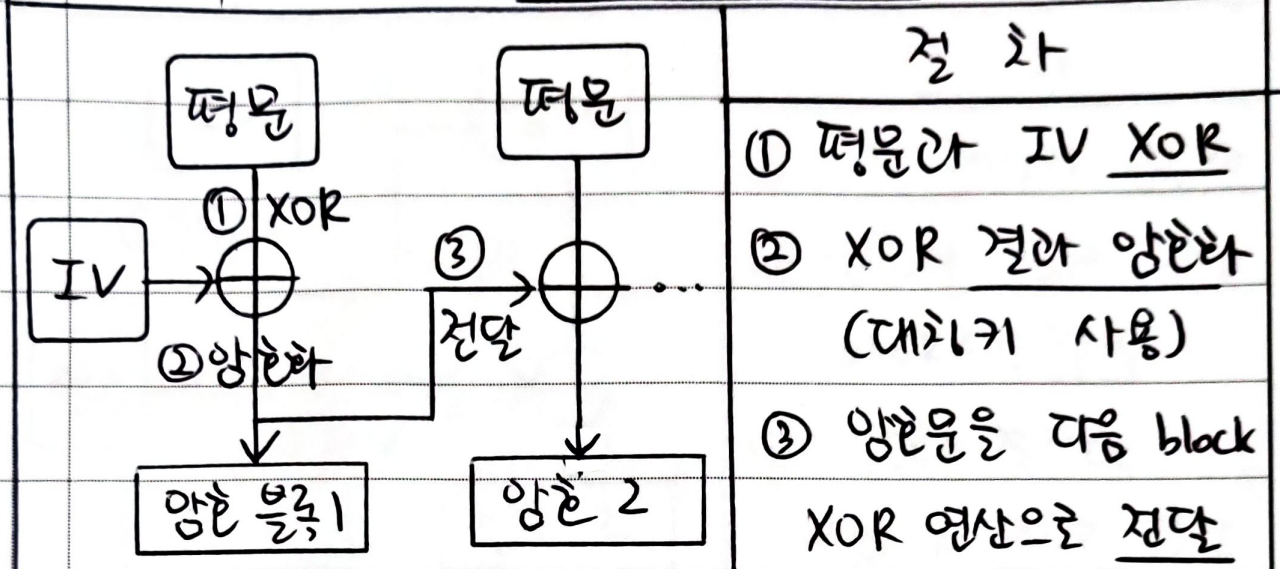
1) ECB (Electronic Code Book)



- ECB는 빠르고 간단하나 패턴 등 유출 가능 한계

2) CBC (Cipher Block Chaining)

개념 : 평문 블록과 IV를 XOR한 후 이를 암호화 해 다음 block 암호화에 활용하는 방식

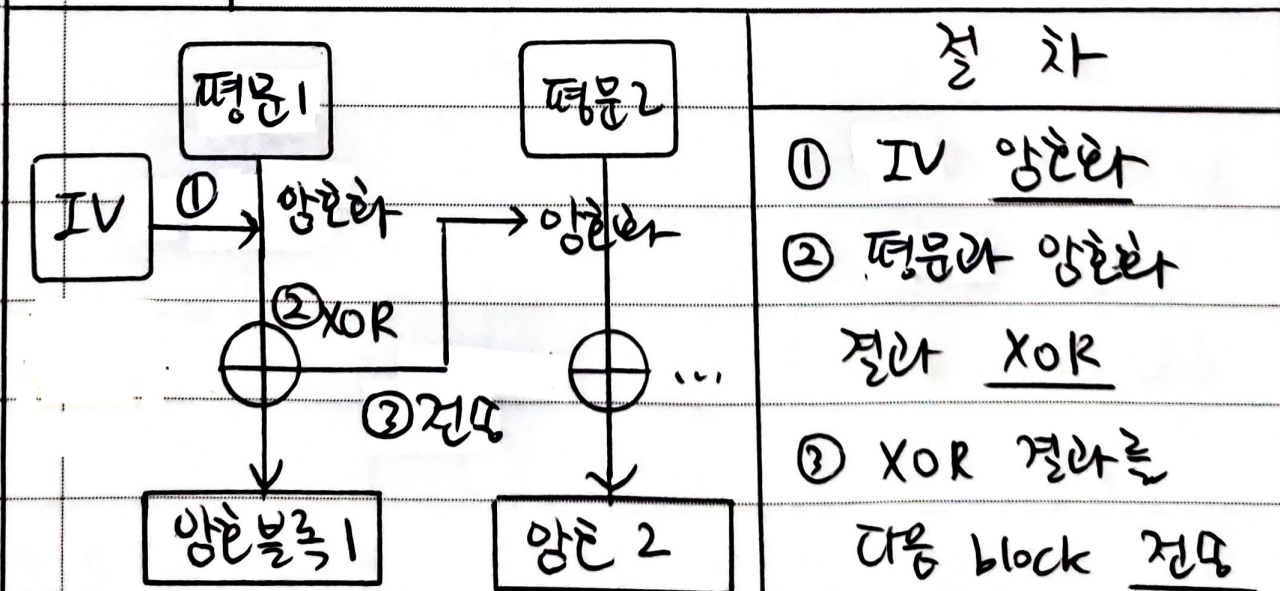


- 평문 블록 1비트만 오류 있어도 모든 암호문 오류 발생.

3. 블록 암호화 응용하는 방식 - CFB, OFB

1) CFB (Cipher Feedback Block)

개념 : 모든 블록 암호를 Stream 암호화 해 각 문자씩 즉시 암호화가 가능한 방식



- 암호화, 복호화 과정 동일, 오류 전파 극복 필요

2) OFB (Output Feedback)

개념	CFB 이 연속 전파 반복하며 block 암호를 stream 암호처럼 변환가능한 모드
	절차 ① IV 채워기 <u>암호화</u> ② 암호화 결과 다음 block에 <u>전파</u> ③ 암호화 결과나 평문 간 <u>XOR</u> 연산

- IV 에 따라 모든 암호문 바뀌므로 CTR과 반복

4. Counter 사용, CTR 모드

개념	암호문 간 독립 연산 수행하나 Feedback 사용하지 않고 Counter 기반의 암호 모드
	절차 ① <u>CTR</u> 암호화 ② 평문과 암호화된 CTR을 <u>XOR</u> - Feedback 없음 - 독립적 연산

- CFB, OFB, CTR을 스트림 암호라 유사하게 사용가능해 바꿀 암호화 시 편함.